

Every declaration splits into halves, and the halves are marked separately. That is the single most useful thing on this page: most people believe a half-written answer is worth nothing, and it is worth half. A blank line scores nothing; half a line scores half. **So always write the half you know.**

1 The bicycle-hire chain [10]

This is Nov 2022 P32 Q4 / June 2025 P31 Q1 / Nov 2025 P31 Q1 wearing different clothes. It comes back.

(a) [2] **MP1** TYPE BikeModel = · **MP2** the five values, bracketed and comma-separated

```
TYPE BikeModel = (Classic, Electric, Cargo, Folding, Tandem)
```

(b) [4] **MP1** TYPE Hire and ENDTYPE both correct · **MP2** DECLARE used for every field · **MP3** any four fields with correct types · **MP4** the remaining field correct, including Model : BikeModel

```
TYPE Hire
  DECLARE HireNumber : STRING
  DECLARE CustomerName : STRING
  DECLARE DateOfHire : DATE
  DECLARE Model : BikeModel
  DECLARE Returned : BOOLEAN
ENDTYPE
```

Field names are your own — only the types are marked. The one to guard is Model : BikeModel, the type you just built in part (a): writing STRING there is the most expensive single error on the question. And DateOfHire must be DATE, not STRING — DATE is in the pseudocode type list.

(c) [1]

```
DECLARE Hire1 : Hire
```

(d) [3] One mark per data category, not per line: **MP1** both strings · **MP2** the date · **MP3** the enumerated value and the Boolean

```
Hire1.HireNumber ← "BH4417"
Hire1.CustomerName ← "R Okafor"
Hire1.DateOfHire ← 03/09/2026
Hire1.Model ← Electric ← no quotes
Hire1.Returned ← FALSE ← no quotes
```

<- is accepted for the arrow. **The dot is required** — Hire1.HireNumber, not HireNumber on its own.

2 Pointers [1 + 1]

```
TYPE TCustomerPointer = ^STRING
TYPE TModelPointer = ^BikeModel
```

Each mark splits as TYPE <name> = · then the correct type it points at.

(b) is the one that catches people, and it is placed after question 1 on purpose. A pointer may point at a type you invented — Nov 2021 P31 Q3(b) asked exactly this, with ^Parts. If you have only ever seen ^INTEGER, this is where you freeze. Worth knowing: in every Paper 3 from 2021 to 2025, a pointer is only ever **declared** — never followed, never dereferenced. There is nothing else to learn.

3 The set [4] — and it is two statements

```
TYPE RowSet = SET OF CHAR
DEFINE ValidRows ('A', 'B', 'C', 'D', 'E') : RowSet
```

MP1 TYPE RowSet = · **MP2** SET OF CHAR · **MP3** DEFINE ValidRows (...) · **MP4** : RowSet

This is the declaration nobody guesses. If you wrote only the TYPE line you did the guessable half and genuinely earned two marks — that is not carelessness, it is the shape of the question. The DEFINE ... : TypeName form has to be *learned*; the only reason anyone knows it is the published mark schemes (Nov 2024 P31 Q6(b), June 2024 P32 Q3(b)). **The quote marks are required** — they are CHAR values.

4 Non-composite [1 each, max 3]

- a data type that is **defined without referencing another data type**
- it may be a **built-in / primitive** type or a **user-defined** one
- a correct **example**: enumerated or pointer, or a valid declaration of one

June 2024 P32 Q3(a), near enough verbatim. **The mark that goes missing is the middle one** — it is tempting to think "non-composite" means "user-defined", when INTEGER is non-composite too. The word is about **how it is built, not about who built it**.

5 Describe a set data type [1 each, max 3]

- a **composite** data type
- holding an **unordered** collection of elements
- **set-theory operations** apply to those elements, such as union and intersection
- the definition includes **the data type of its elements**
- **all elements are of the same type**

Nov 2024 P31 Q6(a). Five acceptable points for three marks, so this is a generous question — but *"it stores a group of things"* scores nothing, and *"it has no duplicates"* on its own does not always score either. **Unordered** and **set-theory operations** are the two that always do.

6 The parallel-array bug [1 + 1]

MP1 — what went wrong. Each member's name is no longer matched to their own fee. The fees moved and the names did not, so the two arrays are out of step and every record after the first change is wrong. **There is no error message because nothing illegal happened:** each array is individually valid, and the connection between them was never written down anywhere the program could see.

MP2 — the fix. A **record**, holding name and fee under one identifier, in a single array — so that sorting moves whole members.

The answer that scores is about the relationship, not about the sort. *"The sort is wrong"* is not it — the sort did exactly what it was asked to do. Ask yourself why no error appeared; that question is the whole lesson.

🌱 **The bit nobody was told about.** The array is 200 long and the club has, say, 130 members. Sorting MemberFee sorts all **200** slots — so the 70 empty ones, holding 0.0, sort to the *front*. Every real member's fee has been shunted 70 places down, past the end of the names that would have matched them. The bug is not just that the data is scrambled; it is that **the empty half of the array joined in**. That is a second, independent reason the two-array design is unsafe, and it is the one that bites in real code.

The five ways this topic is lost

1. **Quoting an enumerated value.** ← "Electric" is wrong. It is a value of a type, not a string.
2. **Declaring the field as STRING** when the previous part just made a type for it. The most expensive mistake on the question.
3. **A set with no DEFINE** — half the marks left behind.
4. **Forgetting a pointer may point at a user-defined type**, not only a built-in one.
5. **Using STRING where DATE belongs.** DATE is in the pseudocode type list and record questions reach for it constantly.

🌱 **How an enumerated type is really stored** — the answer to the "Want to dig?" box on your sheet.

The compiler numbers the values in the order you wrote them — Classic = 0, Electric = 1, Cargo = 2 — so underneath it is an integer after all, which is exactly the thing the lesson argued against. **The storage was never the point.**

With DECLARE Model : INTEGER, the statement Model ← 7 compiles, runs, and produces a bike model that does not exist. With DECLARE Model : BikeModel, **there is no way to write a seventh model** — the compiler rejects the program before it can run.

Identical bits, different set of writable programs. **That is what a type is: not a description of what is in memory, but a restriction on what you are allowed to say about it.**

(And the numbering is not idle — it gives enumerated values an order, which is why Classic < Electric is a legal and meaningful comparison. Cambridge does not examine that; it is simply true.)

🍷 **The Ice-Cream Challenge is still open.** challenge_A01_Records.py — fill in build_class() and sort_by_grade(), and run it until every row is a ✓. **It cannot be faked:** the judge checks that the set of (name, grade) pairs is identical before and after, so if Chen's 58 ends up on Amir you fail, however well sorted the numbers are. 0 marks, 1 ice cream.